www.youtube.com/c/javaexpress
+91 7801007910
Java Express

REST API ASSIGNMENT

Backend Assignment: Build a Category and Product Management System

- **Objective**
  - Create a Spring Boot application using MySQL as the database to manage Category and Product functionalities.
  - The application must include CRUD operations, establish and manage entity relationships, handle exceptions efficiently, and provide detailed endpoint documentation.
  - Additionally, incorporate logging, unit testing, inner joins for optimized data queries, pagination for large datasets, and spring profiles for environment-specific configurations.
- Tech Stack : SpringBoot
- **Database :** MYSQL
- **Features to Implement:**
  - **Category Management**
    - Implement CRUD operations for managing categories.
  - **Product Management:**
    - Implement CRUD operations for managing products.
    - Establish a relationship between products and categories (e.g., Many-to-One relationship).
  - **Exception handling**
    - Implement proper exception handling mechanisms to handle scenarios like:
    - Resource not found (e.g., category or product not found).
    - Validation errors.
  - **Data Models:**
    - **Product**: Fields for id, name, price, description, active, categoryId and createdTime and updatedTime
    - **Category**: Fields for id, name,description, createdTime and updatedTime

www.youtube.com/c/javaexpress
+91 7801007910
Java Express

Category Endpoints

1. **POST /category**
    - **Description**: Create a new category in the system.
    - Request Body : name and description
    - Expected Response :
        i. **201 Created**: Returns the created category with its ID
        ii. **400 Bad Request**: Validation error if mandatory fields are missing or invalid.
2. **PUT /category**
    - Description: Update an existing category using its ID.
    - Request Body: id,name,description
    - Expected Response:
        i. 200 OK: Returns the updated category details.
        ii. 404 Not Found: If the category ID does not exist.
        iii. 400 Bad Request: If input data is invalid.
3. **DELETE /category**
    - Description: Delete an existing category by its ID..
    - Request Parameter:
        i. **Query Parameter**: `id` (required) - ID of the category to delete.
        ii. Example: `/category?id=1`
    - Expected Response:
        i. 200 OK: Returns a success message upon successful deletion
        ii. 404 Not Found: If the category ID does not exist.
4. **GET /category**
    - Description: Retrieve all categories or a specific category by its ID.
    - Request Parameter:
        i. **Optional Query Parameter**: `id` - If provided, fetches details of the specified category.
        ii. Example: `/category?id=1`
    - Expected Response:
        i. 200 OK: if id is provided then same category object else all list of categories

www.youtube.com/c/javaexpress
+91 7801007910
Java Express

Product Endpoints

1. **POST /product**
    - **Description**: Create a new product in the system.
    - Request Body : name,price,description,categoryId
    - Expected Response :
        i. **201 Created**: Returns the created product with its ID
        ii. **400 Bad Request**: Validation error if mandatory fields are missing or invalid categoryid
        iii. **404 Not Found**: If the provided `categoryId` does not exist.
2. **PUT /product**
    - Description: Update an existing product using its ID.
    - Request Body: id, name,price,description,categoryId
    - Expected Response:
        i. 200 OK: Returns the updated product details.
        ii. 404 Not Found: If the product ID or `categoryId` does not exist.
        iii. 400 Bad Request: If input data is invalid.

3. **DELETE / product**
    - **Description**: Delete an existing product by its ID.
    - Request Parameter:
        i. **Query Parameter**: `id` (required) - ID of the category to delete.
        ii. Example: `/product?id=1`
    - Expected Response:
        i. 200 OK: Returns a success message upon successful deletion
        ii. 404 Not Found: If the product ID does not exist.
4. **GET / product**
    - Description: Retrieve all products or a specific product by its ID.
    - Request Parameter:
        i. **Optional Query Parameter**: `id` - If provided, fetches details of the specified category.
        ii. Example: `/product?id=1`
    - Expected Response:
        i. 200 OK:
            - - If an `id` is provided, return the specific product object corresponding to that ID.
            - - If no `id` is provided, return a list of all products.